Data Structures

BS (CS) _Fall_2025

Lab_08 Task

Learning Objectives:

1. Stacks and Queues using Linked Lists in C++

Lab Task 1:

Scenario
You are building a printer software for a shared office printer. Users can submit print jobs with different priorities. Administrative prints should jump ahead of regular user jobs. Your task is to implement a queue to schedule these print jobs.

Task Requirements
Create a priority queue where each print job is an object containing:

- Job ID (integer starting from 1, auto-incremented)
- User Name (string)
- Document Name (string)
- Page Count (integer)
- Priority Level (string: "Admin", "Express", "Normal")

Core Functionalities to Implement

1. **Job Management:**
    - submitJob(userName, documentName, pageCount, priority) - Creates a new job with a unique ID and adds it to the queue at the appropriate position based on the priority level.
    - processJob() - Removes and returns the job from the queue, simulating printing.
2. **Queue Monitoring:**
    - displayQueue() - Shows all jobs in the queue
    - getNextJob() - Views the next job to be processed without removing it.

Advanced Operations

- **Job Cancellation:** cancelJob(jobID) - Removes a specific job from the queue.
- **Priority Update:** changeJobPriority(jobID, newPriority) - Changes the priority of an existing job and reorder the queue.

Implementation Guidelines

- Implement a PrintJob class and a PrinterQueue class.
- The priority can be managed by assigning integer weights to priority levels (e.g., Admin = 3, Express = 2, Normal = 1).

Google Test Cases:

```
TEST(PrinterQueueTest, SubmitJobAssignsUniqueIDs) {
    PrinterQueue<int> pq;
    int id1 = pq.submitJob("Alice", "Doc1", 5, "Normal");
    int id2 = pq.submitJob("Bob", "Doc2", 3, "Normal");
    EXPECT_EQ(id1, 1);
    EXPECT_EQ(id2, 2);
}

TEST(PrinterQueueTest, SubmitJobStoresCorrectDetails) {
    PrinterQueue<int> pq;
    int id = pq.submitJob("Charlie", "Report", 10, "Express");
    PrintJob<int> job = pq.getNextJob();
    EXPECT_EQ(job.jobID, id);
    EXPECT_EQ(job.userName, "Charlie");
    EXPECT_EQ(job.documentName, "Report");
    EXPECT_EQ(job.pageCount, 10);
    EXPECT_EQ(job.priority, "Express");
}

TEST(PrinterQueueTest, AdminJobsJumpAhead) {
    PrinterQueue<int> pq;
    pq.submitJob("User1", "File1", 4, "Normal");
    pq.submitJob("User2", "File2", 2, "Express");
    pq.submitJob("AdminUser", "AdminDoc", 1, "Admin");

    PrintJob<int> job = pq.getNextJob();
    EXPECT_EQ(job.priority, "Admin");
    EXPECT_EQ(job.userName, "AdminUser");
}

TEST(PrinterQueueTest, ExpressJobsBeforeNormal) {
    PrinterQueue<int> pq;
    pq.submitJob("User1", "File1", 4, "Normal");
    pq.submitJob("User2", "File2", 2, "Express");

    PrintJob<int> job = pq.getNextJob();
    EXPECT_EQ(job.priority, "Express");
    EXPECT_EQ(job.userName, "User2");
}

TEST(PrinterQueueTest, ProcessJobRemovesFromQueue) {
    PrinterQueue<int> pq;
    pq.submitJob("Alice", "Doc1", 2, "Normal");
```

```
    pq.submitJob("Admin", "Doc2", 1, "Admin");

    PrintJob<int> job = pq.processJob();
    EXPECT_EQ(job.priority, "Admin");

    PrintJob<int> next = pq.getNextJob();
    EXPECT_EQ(next.userName, "Alice");
}

TEST(PrinterQueueTest, CancelJobRemovesJob) {
    PrinterQueue<int> pq;
    int id1 = pq.submitJob("User1", "Doc1", 5, "Normal");
    int id2 = pq.submitJob("User2", "Doc2", 3, "Express");

    EXPECT_TRUE(pq.cancelJob(id1));

    PrintJob<int> job = pq.processJob();
    EXPECT_EQ(job.jobID, id3);
    EXPECT_EQ(job.userName, "User2");
    EXPECT_EQ(job.documentName, "Doc2");
    EXPECT_EQ(job.pageCount, 3);
    EXPECT_EQ(job.priority, "Express");

}

TEST(PrinterQueueTest, CancelNonExistingJobReturnsFalse) {
    PrinterQueue<int> pq;
    pq.submitJob("User1", "Doc1", 2, "Normal");
    EXPECT_FALSE(pq.cancelJob(99));
}

TEST(PrinterQueueTest, ChangePriorityReordersQueue) {
    PrinterQueue<int> pq;
    int id1 = pq.submitJob("User1", "Doc1", 5, "Normal");
    int id2 = pq.submitJob("User2", "Doc2", 3, "Normal");

    EXPECT_TRUE(pq.changeJobPriority(id1, "Admin"));

    PrintJob<int> next = pq.processJob();
    EXPECT_EQ(next.jobID, id1);
    EXPECT_EQ(next.priority, "Admin");
}

TEST(PrinterQueueTest, ChangePriorityOfNonExistingJobReturnsFalse) {
    PrinterQueue<int> pq;
```

```
    pq.submitJob("User1", "Doc1", 2, "Normal");
    EXPECT_FALSE(pq.changeJobPriority(42, "Admin"));
}

TEST(PrinterQueueTest, SubmitJobWithZeroPagesAllowed) {
    PrinterQueue<int> pq;
    int id = pq.submitJob("User", "EmptyDoc", 0, "Normal");
    PrintJob<int> job = pq.processJob();
    EXPECT_EQ(job.pageCount, 0);
    EXPECT_EQ(job.jobID, id);
}
```

Lab Task 2

Scenario

Programming languages use a call stack to manage function calls and returns. When a function is called, a stack frame is pushed. When a function returns, its frame is popped. Your task is to simulate this behavior.

Task Requirements

Create a stack where each stack frame is an object containing:

- Function Name (string)
- Line Number of Call (integer, to simulate execution order)
- Number of Parameters(integer)

Core Functionalities to Implement

1. **Execution Simulation:**
    - callFunction() - Pushes a new stack frame for the function onto the call stack.
    - returnFromFunction() - Pops the top stack frame from the call stack, simulating a function return.
2. **Stack Inspection:**
    - displayCallStack() - Displays the current call stack from top (most recent) to bottom (main), showing the function name and call order.
    - getCurrentFunction() - Returns the name of the function at the top of the stack (the currently executing function).

Implementation Guidelines

- Implement a Function and StackFrame class

- You StackFrame class should also have a maximum limit to return false while insertion to simulate a stack overflow error.

Google Test Cases:

```
TEST(CallStackTest, CallSingleFunction) {
    CallStack<int> stack(5); // max size 5
    EXPECT_TRUE(stack.callFunction("main", 1, 0));
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}

TEST(CallStackTest, CallMultipleFunctions) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 5, 2);
    stack.callFunction("funcB", 10, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcB");
}

TEST(CallStackTest, ReturnFromFunction) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 2, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcA");
    EXPECT_TRUE(stack.returnFromFunction());
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}

TEST(CallStackTest, ReturnFromEmptyStack) {
    CallStack<int> stack(5);
    EXPECT_FALSE(stack.returnFromFunction());
}

TEST(CallStackTest, StackOverflow) {
    CallStack<int> stack(2); // max size 2
    EXPECT_TRUE(stack.callFunction("main", 1, 0));
    EXPECT_TRUE(stack.callFunction("funcA", 2, 1));
    EXPECT_FALSE(stack.callFunction("funcB", 3, 2)); // should fail
}

TEST(CallStackTest, GetCurrentFunctionEmpty) {
    CallStack<int> stack(5);
    EXPECT_EQ(stack.getCurrentFunction(), ""); // No function should be running
```

```
}

TEST(CallStackTest, NestedFunctionCallsAndReturns) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 2, 2);
    stack.callFunction("funcB", 5, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcB");
    stack.returnFromFunction(); // returning from funcB
    EXPECT_EQ(stack.getCurrentFunction(), "funcA");

    stack.callFunction("funcC", 7, 3);
    EXPECT_EQ(stack.getCurrentFunction(), "funcC");

    stack.returnFromFunction(); // returning from funcC
    EXPECT_EQ(stack.getCurrentFunction(), "funcA");

    stack.returnFromFunction(); // returning from funcA
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}
```